תרגיל בית 3 – MDP, RL ומבוא ללמידה

עברו על כלל ההנחיות לפני תחילת התרגיל.

הנחיות כלליות:

- תאריך ההגשה: עד ל-29/01/2026 בשעה 23:59
- את המטלה יש להגיש **בזוגות בלבד.**
- יש להגיש מטלות מוקלדות בלבד. פתרונות בכתב יד לא ייבדקו.
- ניתן לשלוח שאלות בנוגע לתרגיל בפיאצה בלבד.
- האחראי על תרגיל זה: **נבו שניידר**
- בקשות דחיה מוצדקות (מילואים, אשפוז וכו') יש לשלוח למתרגל האחראי (**רועי טייכמן**) בלבד.
- במהלך התרגיל ייתכן שנעלה עדכונים מחייבים בפיאצה, עליכם להתעדכן בהם עד מועד הגשת התרגיל.
- שימו לב, העתקות תטופלנה בחומרה.
- התשובות לסעיפים בהם מופיע הסימון ✍️ צריכים להופיע בדוח.
- לחלק הרטוב מסופק שלד של הקוד.
- אנחנו קשובים לפניות שלכם במהלך התרגיל ומעדכנים את המסמך הזה בהתאם. גרסאות עדכניות של המסמך יועלו לאתר. הבהרות ועדכונים שנוספים אחרי הפרסום הראשוני יסומנו כאן בצבעים. ייתכן שתפורסמנה גרסאות רבות – אל תיבהלו מכך. השינויים בכל גרסה יכולים להיות קטנים.

שימו לב שאתם משתמשים רק בספריות הפייתון המאושרות בתרגיל (מצוינות בתחילת כל חלק רטוב)
לא יתקבל קוד עם ספריות נוספות

מומלץ לחזור על שקפי ההרצאות והתרגולים הרלוונטיים לפני תחילת העבודה על התרגיל.

חלק א' – MDP (67 נק')

רקע

בחלק זה נעסוק בתהליכי החלטה מרקובים, נתעניין בתהליך עם **אופק אינסופי** (מדיניות סטציונרית).

חלק א' - חלק היבש ✍️

שאלה 1 - 10 נקודות

בתרגול ראינו את משוואת בלמן כאשר התגמול ניתן עבור המצב הנוכחי בלבד, כלומר $R: S \to R$,
למתן תגמול זה נקרא "תגמול על הצמתים" מכיוון שהוא תלוי בצומת שהסוכן נמצא בו.
בהתאם להגדרה זו הצגנו בתרגול את האלגוריתמים Value iteration ו-Policy Iteration למציאת המדיניות האופטימלית.

כעת, נרחיב את ההגדרה הזו, לתגמול המקבל את המצב לפני הפעולה, המצב אחרי הפעולה, והפעולה עצמה, כלומר: $R: S \times S \times A \to R$, למתן תגמול זה נקרא "תגמול תהליכי". לצורך שלמות ההגדרה, נגדיר שאם לכל
$a \in A$ מתקיים - $P(s, s', a) = 0$ אז $R(s, s', a') = -\infty$.

1. (2 נק') התאימו את הנוסחה של התוחלת של התועלת מהתרגול, עבור התוחלת של התועלת המתקבלת במקרה של " תגמול תהליכי", אין צורך לנמק.

$$U^{\pi}(s) = E_{\pi}[\sum_{t=0} \gamma^t R(s_t, s_{t+1}) | s_0 = s]$$

2. (2 נק') כתבו מחדש את נוסחת משוואת בלמן עבור המקרה של " תגמול תהליכי", אין צורך לנמק.

$$\boxed{U^*(s) = \max_{a \in A} \sum_{s' \in S} P(s'|s, a)[R(s, a, s') + \gamma U^*(s')]}$$

בסעיפים הבאים התייחסו גם למקרה בו $\gamma = 1$, והסבירו מה לדעתכם התנאים שצריכים להתקיים על הסביבה\mdp על מנת שתמיד נצליח למצוא את המדיניות האופטימלית.

3. 3) נק') נסחו את אלגוריתם Value Iteration עבור המקרה של " תגמול תהליכי".

For $\gamma = 1$, the conditions that must hold on the MDP so that we are guaranteed to find the optimal policy are: a finite state space S, reward must be bounded $|R(s,a,s')|<\infty$ so we can prevent utilities from diverging to infinity and there are no cycles that yield positive reward forever without discounting , otherwise the optimal value may be $+\infty$, and Value Iteration will not converge

inputs:

- mdp: an MDP with
  - states S
  - actions $A(s)$
  - transition model $P(s' \mid s, a)$
  - **process reward** $R(s, a, s')$
  - discount factor γ
- ϵ: maximum allowed error in the utility of any state

local variables:

- U, U': utility vectors for states in S, initially zero
- δ: maximum change in the utility of any state in one iteration

repeat

$U \leftarrow U'$

$\delta \leftarrow 0$

for each state $s \in S$ **do**

$$U'(s) \leftarrow \max_{a \in A(s)} \sum_{s' \in S} P(s' \mid s, a)[R(s, a, s') + \gamma U(s')]$$

if $| U'(s) - U(s) | > \delta$ **then**

$\delta \leftarrow | U'(s) - U(s) |$

until

$$\delta < \frac{\epsilon(1-\gamma)}{\gamma}$$

return U

4. 3) נק') נסחו את אלגוריתם Policy Iteration עבור המקרה של " תגמול תהליכי".

For $\gamma = 1$, Policy Iteration may not stop because policy evaluation does not converge, so the stopping condition (unchanged policy) is never satisfied.

Policy Iteration with Process Reward

function POLICY-ITERATION(mdp) **returns a policy**

inputs:

- mdp: an MDP with
  - states S
  - actions $A(s)$
  - transition model $P(s' \mid s, a)$
  - **process reward** $R(s, a, s')$
  - discount factor γ

local variables:

- U: a vector of utilities for states in S, initially zero
- π: a policy vector indexed by state, initially random

repeat

1. **Policy Evaluation**
   Compute $U = U^{\pi}$ such that for every $s \in S$:

$$U(s) = \sum_{s' \in S} P(s' \mid s, \pi(s))[R(s, \pi(s), s') + \gamma U(s')]$$

2. **Policy Improvement**

unchanged? ← true

for each state $s \in S$**do**

$$\text{if } \max_{a \in A(s)} \sum_{s' \in S} P(s' \mid s,a)[R(s,a,s') + \gamma U(s')] > \sum_{s' \in S} P(s' \mid s,\pi(s))[R(s,\pi(s),s') + \gamma U(s')]$$

then

$$\pi(s) \leftarrow \arg \max_{a \in A(s)} \sum_{s' \in S} P(s' \mid s,a)[R(s,a,s') + \gamma U(s')]$$

unchanged? ← false

until unchanged

return π

<u>שאלה 2 - 12 נקודות</u>

זואי החתולה מבקשת מהוריה האנושיים חטיפים. בכל רגע היא יכולה להסתפק באוכל שקיבלה ולבקש עוד מחר, או לבקש עוד. בהסתברות p, הוריה יסכימו לתת לה עוד אוכל (מנה של חטיף, ניתן להתייחס אל זה כאל המספר 1), ובהסתברות $1-p$, הוריה יסרבו לבקשותיה ולא יתנו לה עוד חטיפים למשך שבעת הימים הבאים.

- זואי לא מקבלת חטיפים מבלי שהיא ביקשה חטיף והוריה נתנו לה
- בכל מקרה ומקרה שבו זואי עוברת ליום הבא קיימת דעיבה של התועלת
- זואי יכולה על פניו לבקש אינסוף פעמים באותו יום

- ישנה דעיבה בתועלת בין הימים של 0.9, כלומר, זואי תעדיף אוכל עכשיו על פני אוכל מחר, ובטח ובטח על פני האוכל מחרתיים או בסופשבוע, אך אין דעיבה כזו באותו היום.
- ביום עצמו יש אופק סופי "גדול מאוד", כלומר, בקירוב של פיזיקאים ניתן להניח שהוא אינסופי.
- זואי רוצה לקבל כמה שיותר אוכל.
- התייחסו בפתרונכם לשבעת הימים הבאים כולל יום זה.

1. (2 נק') נסחו נא בעיה זו כבעיית MDP עם מצבים הממוספרים מ 0 עד n ומצב סיום T.

States:

State 0: where Zoe is allowed to ask for another snack

State 1 -> 7 : the waiting days for the penalty to end

State T: terminal state, representing the end of the asking(maybe Zoe got adopted by a nicer family)

A(s) the action group for each s:

In state 0:

Ask : attempt to get another snack now

Wait : stop asking and move to the next day

In states 1–7:

Wait : the only possible action, since the parents refuse requests during the penalty

In state T:

no actions (terminal state)

Transition Probabilities and Rewards:

$p =$ be the probability that the parents agree

$q = 1 - p$ be the probability that they refuse

With a discount reward from day to day $\gamma^t R_t(s)$

Nodes (States)

- State 0 → Zoe can ask for a snack or wait.
- States 1–7 → Cooldown days (cannot ask, only wait).

Transitions (the process is only for 8 days so $0 \leq t \leq 7$)

1. From State 0:
    - Ask → State 0 with probability p, reward = $\gamma^t * 1$, $t \leftarrow t$ (parents agree and Zoe stays in the same day).
    - Ask → State 7 with probability $1 - p$, reward = $\gamma^t * 0 = 0$ (parents refuse → cooldown).
    - Wait → State 0 with probability 1, reward = $\gamma^t * 0 = 0$ (skip day $t \leftarrow t + 1$).
2. From State s ∈ {1, …, 7}:
    - Wait → State s-1 with probability 1, reward = $\gamma^t * 0 = 0$ (cooldown reduces by 1 day).

Discount Factor

- Discounting occurs only between days
- Within state 0, when the action Ask and the system remains in state 0 so no discount is applied ($\gamma = 1$), since we stay in the same day
- Whenever a day passes (transition from $s$to $s - 1$, or choosing Wait in state 0), a discount factor of $\gamma = 0.9$ is applied

2. (2 נק') האם ניתן לנסח בעיה זו כבעיה עם שני מצבים בלבד?

No because it will no longer be an MDP problem , in MDP problems the next state depends only on the current state and actions and it doesn't depend on the history of actions , if we want to do it in 2 states we will need to have a s0 state and encode the "waiting for 7 days " in the transitions rather than into the state space in the s1 state.

3. (3 נק') האם ניתן לנסח בעיה זו באשר התגמולים שליליים?

Yes the problem can be changed using negative rewards by defining a cost for each snack, so that the goal is to minimize the total cost, the new problem will be equivalent to the original one and does not change the optimal policy

4. (5 נק') נניח כעת שאנו מסתכלים רק על 3 ימים, ועל 4 "רגעי ביקוש" בכל יום. אנא מכם, עזרו לזואי להחליט על מדיניות אופטימלית בכל "רגע ביקוש", כתלות בהסתברות p.

We will solve it using policy iteration :
We consider the state space

$$S = \{s_{d,k} \mid d \in \{1,2,3\},\ k \in \{1,2,3,4\}\} \cup \{T\}$$

where $s_{d,k}$denotes day $d$and demand moment k, and $T$is a terminal state.
For every $s \neq T$, the action set is

$$A(s) = \{\text{Ask,Wait}\}.$$

initial policy
We start with a cautious policy : $\pi 0(s) = Wait\ \forall s \neq T$
Policy evaluation
Under π_0, Zoe never asks for snacks, so she receives zero reward:

$$V^{\pi_0}(s) = 0 \forall s.$$

Policy improvement
For any state s:

$$U^{\pi_0}(s,\text{Ask}) = p \cdot 1 + (1-p) \cdot 0 = p,$$
$$U^{\pi_0}(s,\text{Wait}) = 0.$$

For all $p > 0$, $U^{\pi_0}(s,\text{Ask}) > U^{\pi_0}(s,\text{Wait})$, therefore the improved policy is

$$\pi_1(s) = \text{Ask}\ \forall s \neq T.$$

Policy evaluation of $\boldsymbol{\pi_1}$
All non-terminal states are equivalent, so let $V^{\pi_1}(s) = V$.
The Bellman equation is

$$V = p(1+V).$$

Thus,

$$V = \frac{p}{1-p}.$$

Policy improvement check

$$U^{\pi_1}(s,\text{Ask}) = p(1+V) = \frac{p}{1-p},$$
$$U^{\pi_1}(s,\text{Wait}) = \gamma V \leq 0.9\frac{p}{1-p}.$$

Hence $U^{\pi_1}(s,\text{Ask}) > U^{\pi_1}(s,\text{Wait})$, so the policy is stable.

An easier way to think about the problem is :
Waiting only delays a reward that is already available now,
and delayed rewards are discounted.
Therefore Optimal policy:

Ask at every demand moment, For all days, all moments

חלק ב' - היכרות עם הקוד

חלק זה הוא רק עבור היכרות הקוד, עבורו עליו במלואו וודאו כי הינכם מבינים את הקוד.

- mdp.py **אתם לא צריכים לערוך כלל את הקובץ הזה**.

בקובץ זה ממומשת הסביבה של ה-mdp בתוך מחלקת MDP. הבנאי מקבל:

- board - המגדיר את המצבים האפשריים במרחב ואת התגמול לכל מצב, תגמול על הצמתים בלבד.
- terminal_states – קבוצה של המצבים הסופיים (בהכרח יש לפחות מצב אחד סופי).
- transition_function – מודל המעבר בהינתן פעולה, מה ההסתברות לכל אחת מארבע הפעולות האחרות. ההסתברויות מסודרות לפי סדר הפעולות.
- gamma – discount factor המקבל ערכים - $\gamma \in (0,1)$.
  בתרגיל זה לא נבדוק את המקרה בו $\gamma = 1$.

הערה: קבוצת הפעולות מוגדרת בבנאי והיא קבועה לכל לוח שיבחר.

למחלקת MDP יש מספר פונקציות שעשויות לשמש אתכם בתרגיל.

- print_rewards() – מדפיסה את הלוח עם ערך התגמול בכל מצב.

- print_utility(U) – מדפיסה את הלוח עם ערך התועלת U לכל מצב.
- print_policy(policy) – מדפיסה את הלוח עם הפעולה שהמדיניות policy נתנה לכל מצב שהוא לא מצב סופי.
- step(state, action) – בהינתן מצב נוכחי state ופעולה action מחזיר את המצב הבא באופן דטרמיניסטי. עבור הליכה לכיוון קיר או יציאה מהלוח הפונקציה תחזיר את המצב הנוכחי state.
- load_mdp(board,termina_states,transition_function,gamma) – פונקציה סטטית המקבלת שמות של קבצים המכילים את מבנה הmdp, ואת ערכה של gamma ומחזירה אוביקט mdp.

הערה: פונקציות ההדפסה שלנו משתמשות בספרית פייתון בשם termcolor בשביל לסמן מצבים סופיים באדום וקירות בכחול. אנו ממליצים להשתמש ב־integrated terminal שנמצא בIDE שלכם על מנת לראות את ההדפסות בפורמט ברור.

להלן דוגמה להדפסה:

```
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@ The board and rewards @@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
| -0.04 | -0.04 | -0.04 | +1    |
| -0.04 | WALL  | -0.04 | -1    |
| -0.04 | -0.04 | -0.04 | -0.04 |
```

חלק ג' – רטוב

כל הקוד צריך להיכתב בקובץ mdp_rl_implementation.py

מותר להשתמש בספריות:

All the built-in packages in python, numpy, matplotlib, argparse, os, copy, typing, termcolor, random

עליכם לממש את הפונקציות הבאות:

- (רטוב 7 נק'): value_iteration(mdp, U_init, epsilon) – בהינתן ה-mdp, ערך התועלת ההתחלתי U_init, וחסם העליון לשגיאה מהתוחלת של התועלת האופטמילי epsilon מריץ את האלגוריתם value iteration ומחזיר את U המתקבל בסוף ריצת האלגוריתם. הערך עבור קירות הינו None. **TODO**
- (רטוב 7 נק'): get_policy(mdp, U) – בהינתן ה-mdp וערך התועלת U (המקיים את משוואת בלמן) מחזיר את המדיניות (במידה וקיימת יותר מאחת, מחזיר אחת מהן). המדיניות עבור קירות ומצבים סופיים הינה None. **TODO**
- (רטוב 7 נק'): policy_evaluation(mdp, policy) – בהינתן ה-mdp, ומדיניות policy מחזיר את ערכי התועלת לכל מצב. הערך עבור קירות הינו None. **TODO**
- (רטוב 7 נק'): policy_iteration(mdp, policy_init) – בהינתן ה-mdp, ומדיניות התחלתית policy_init, מריץ את האלגוריתם policy iteration ומחזיר מדיניות אופטימלית. המדיניות עבור קירות ומצבים סופיים הינה None. **TODO**

הבהרה לחלק זה של התרגיל - הסימולציה של המדיניות כבר נערכה, ומסופק לכם קובץ simulation_results בהתאם המכיל את תוצאותיה ביחד עם הפעולות שניסינו לבצע והפעולות שביצענו בפועל בכל צעד. עליכם לממש קריאה ל'סימולטור' (שהוא בפועל parser של קובץ התוצאות) ואז לקרוא את האפיזודה שבוצעה ובהמשך לממש את האלגוריתם כפי שנלמד בכיתה.

אבוי! נבו, המתרגל ה"אחראי" על התרגיל, איבד את קבצי ה־board וה־transition_function ובעת אין אנו יודעים את R ואת P. לנבו יש מדיניות π שאותה הוא רוצה להעריך, כלומר לחשב את U^{π}, אך בעת אינו יכול לעשות זאת. למזלו, הראל - מומחה ה־RL™ , סיפר לו שהוא יודע לבצע **<u>סימולציות</u>**, כלומר לבצע רצף של פעולות אקראיות במרחב ה־MDP ועל הדרך לתעד את ה־Rewards המתקבלים, הפעולות שנבחרו והפעולות שהתבצעו בפועל. סימולציה אחת, אשר נקראת גם אפיזודה (episode), מוגבלת עד ל־k צעדים ומורכבת מרצפים מהצורה:

$$state, reward, action, actual\ action$$

בצעד האחרון נבצע שני הפעולות יהיו $None$.

הראל הציע לנבו להשתמש באלגוריתם Passive RL – model free, כלומר להשתמש בסימולציות על מנת ללמוד **<u>ולהעריך</u>** את התוחלת של התועלת עבור המדיניות שאותה נבו רוצה להעריך.

ספציפית הראל הציע לנבו להשתמש באלגוריתם First-visit Monte-Carlo Estimation.

לדוגמה, עבור מקדם דעיכה- $\gamma = 1$, אם הרצנו אפיזודה אחת בלבד וקיבלנו:

$$A, 3, Down, Up$$

$$B, 2, Up, Down$$

$$A, 3, Down, Down$$

$$C, 5, Up, Up$$

$$A, 3, None, None$$

מהרצת first visit MC:

$$V_\pi(A) = 16, V_\pi(B) = 13, V_\pi(C) = 8$$

אלגוריתם זה נקרא Monte Carlo estimation.

סיפקנו לכם בקובץ simulator.py סימולטור שיאפשר לכם לממש את האלגוריתם, אנא קראו אותו וראו דוגמת הרצה.

- (רטוב 7 נק') ממשו פונקציה:

– (mc_algorithm(sim, num_episodes, gamma ,num_rows, num_cols, actions, policy בהינתן הסימולטור, מספר האפיזודות שיש להריץ, מספר השורות, מספר העמודות ופוליסה לשערך, מריצה את האלגוריתם **First-visit** Monte Carlo estimation למשך num_episodes אפיזודות ומחזיר את estimated utility אשר התקבל בסיום הריצה. התגמול עבור קירות הינו $None$. אם לא נתקלתם במצב מסוים באף אחת מהאפיזודות, הניחו שערך התוחלת של התועלת עבורו הינו 0. **TODO**

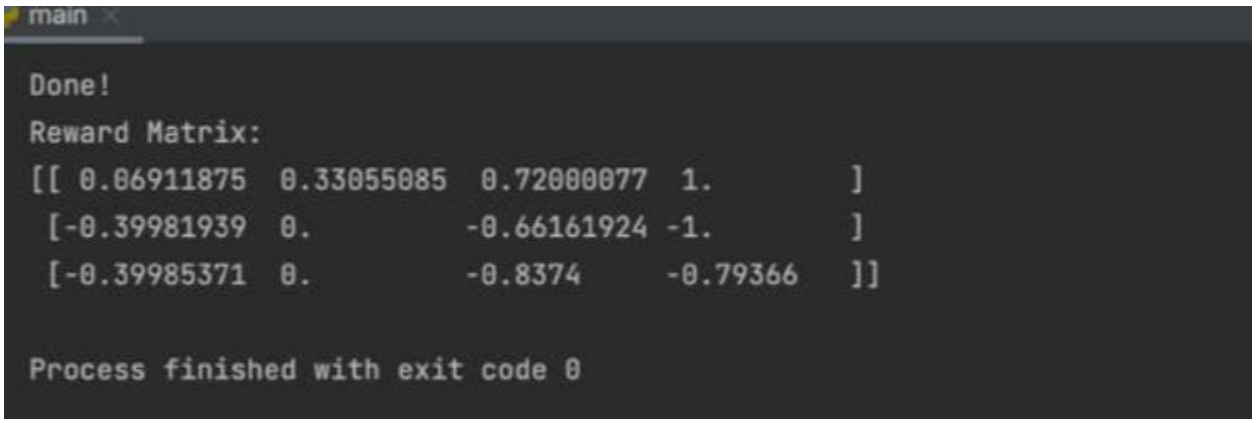
תזכורת: אנו בגישת First-Visit, ולכן הקפידו לעדכן את ערכי התגמול רק בפעם הראשונה בה נתקלתם במצב באפיזודה מסוימת.

- ✍️ (יבש 3 נק') הריצו את ה mc_algorithm שלוש פעמים עם num_episodes=10, 100, 1000. צרפו את תוצאות ההרצה. האם התוצאות זהות? איזו תוצאה קרובה יותר לשערוך האמיתי של המדיניות? מה היתרונות והחסרונות של הגדלת מספר ה episodes?

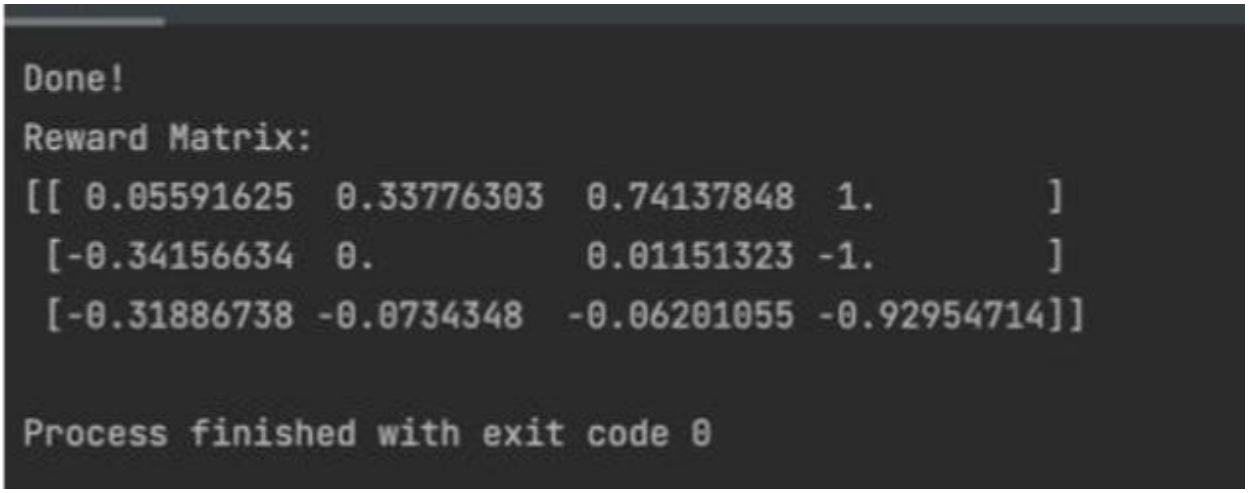
Among the three runs, the result obtained with **1000 episodes** is the closest to the true value of the policy,on one hand Increasing the number of episodes stabilizes the estimates which leads to convergence toward the true policy value.

On the other hand, it increases computational cost which requires more interaction with the environment and leads to slow learning since updates are performed only after complete episodes.

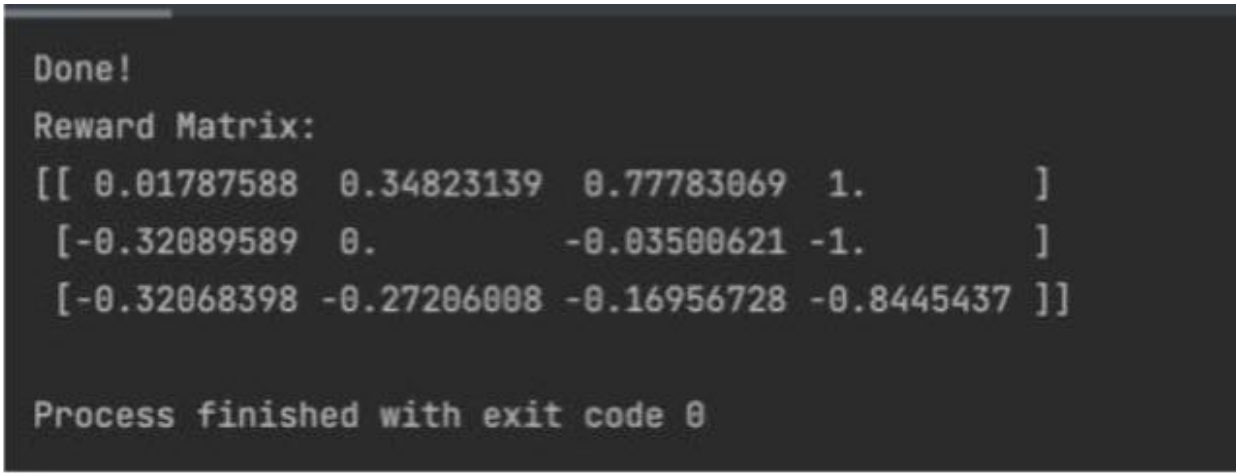
Num_episodes = 10:



Num of episodes = 100:



Num of episodes = 1000:



עבור הסעיפים הבאים נרצה להתאים את אלגוריתם First-Visit Monte Carlo ל־MDPs עם מסלולים אינסופיים.

יהי מודל החלטה מרקובי (MDP עם $M = (S, A, P, R, \gamma)$) $\gamma \in (0,1)$ ומודל תגמול על המצבים $R: S \to R$.
קיים חסם R_{max} על הערכים המוחלטים של התגמולים, כלומר: $\forall s \in S: |R(s)| \leq R_{max}$.

עבור מדיניות π אותה נרצה לשערך, תהיינה $N\{\tau_i\}_{i=1}^{N}$ סימולציות **<u>אשר כולן מתחילות ממצב התחלתי s_0.</u>**
כל הסימולציות הינן אינסופיות. לכל $i \in \{1, \dots, N\}$ ולכל $t \in N$ נסמן בs_t^i את המצב ה-t בסימולציה ה-i .

- ✍️(יבש 2 נק') השלימו את הביטוי עבור $V_\infty^\pi(s_0)$, המייצג את ערך התוחלת של התועלת על פי המדיניות π אשר היה משוערך ע"י אלגוריתם First-Visit Monte Carlo אם היינו יכולים להריץ אותו על סימולציות אינסופיות:

$$V_\infty^\pi(s_0) = \frac{1}{N} \sum_{i=1}^{N} \quad \sum_{t=0}^{\infty} \gamma^t \, R(s_i^t)$$

לצערנו אין אנו יכולים לסכום אינסוף איברים ולכן נצטרך להגביל את הסכימה בכל סימולציה ל-T תגמולים בלבד. נסמן ב־$V_T^\pi(s_0)$ את ערך התועלת על פי המדיניות π אשר היה משוערך ע"י האלגוריתם אם היינו מגבילים כל סימולציה ל-T מצבים בלבד.

- ✍️(יבש 3 נק') יהי $\epsilon > 0$, מצאו חסם תחתון ל-T עבורו הרצת First-Visit Monte Carlo תחזיר ערך $V_T^\pi(s_0)$, המקיים: $|V_T^\pi(s_0) - V_\pi(s_0)| \leq \epsilon$. ערך זה יכול להיות תלוי ב- $\epsilon, R_{max}, \gamma$.

ניתן להניח שמתקיים עבור π $|V_{\infty}^{\pi}(s_0) - V_{\pi}(s_0)| \cong 0$:. הוכיחו נכונות של החסם שמצאתם.

רמז: התחילו מלמצוא חסם עליון הדוק ל- $|V_{\infty}^{\pi}(s_0) - V_{T}^{\pi}(s_0)|$.

$$V_T^{\pi}(s_0) = \frac{1}{N}\sum_{i=1}^{N} \sum_{t=0}^{T-1} \gamma^t R(s_i^t)$$

$$|V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0)| = |\frac{1}{N}\sum_{i=1}^{N}\sum_{t=T}^{\infty} \gamma^t R(s_i^t)|$$

We know $|R(s)| \leq R_{\max}$, so:

$$|V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0)| \leq \sum_{t=T}^{\infty} \gamma^t R_{max} = R_{max} \sum_{t=T}^{\infty} \gamma^t$$

So we have the tight upper bound:

$$|V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0)| \leq Rmax * \frac{\gamma T}{(1-\gamma)}$$

We want $|V_T^{\pi}(s_0) - V_{\pi}(s_0)| \leq \varepsilon$. Assuming $|V_{\infty}^{\pi}(s_0) - V_{\pi}(s_0)| \simeq 0$, it's enough to satisfy:

$$|V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0)| \leq \varepsilon$$

$$|V_T^{\pi}(s_0) - V_{\pi}(s_0)| = |V_T^{\pi}(s_0) - V_{\pi}(s_0) + V_{\infty}^{\pi}(s_0) - V_{\infty}^{\pi}(s_0)| = |V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0)| \leq \epsilon$$

Now we plug in the upper bound

$$Rmax * \frac{\gamma T}{(1-\gamma)} \leq \epsilon$$

And we get …

$$\boxed{T \geq \frac{ln\,(\varepsilon(1-\gamma)/R_{max})}{ln\,\gamma}}$$

- ✍️ (יבש 2 נק') בהתבסס על הסעיף הקודם, הציעו אלגוריתם anytime עבור בעיית first-visit MC ל־MDPs עם מסלולים אינסופיים. הסבירו מדוע האלגוריתם שהצעתם מתיישב עם גישת ה־anytime כפי שנלמדה בקורס.

The anytime property means: even if you stop before reaching your planned T, the estimate is still valid — it's just less accurate.

$$| V_{\infty}^{\pi}(s_0) - V_T^{\pi}(s_0) | \leq \varepsilon$$

If we computed $V_T^{\pi}(s_0)$ we will get an error smaller than ϵ , with the T that already been calculated :

The algorithm:

- Initialize $V(s) = 0$and Returns(s)=[] for all states
- Generate episodes from $s_0$following policy π, **up to T steps**
- For each state **first visited** in an episode, compute truncated return $G_T = \sum_{t=0}^{T-1} \gamma^t R(s_t)$and add it to Returns(s).
- Update $V(s) =$ average of Returns(s); increase T gradually to improve accuracy.
- **Anytime:** stop at any moment and return current $V(s_0)$; longer run → closer to true value, The longer you run (larger T or more episodes), the closer $V(s_0)$gets to the true value

חלק ב' - מבוא ללמידה (28 נק')

✍️ חלק א' – חלק היבש (7 נק')

מתפצלים ונהנים

(7 נק') בידוע, בעת סיווג של דוגמת מבחן על ידי עץ החלטה, בכל צומת בעץ אנו מחליטים לאיזה צומת בן להעביר את דוגמת המבחן על ידי ערך סף v שמושווה לfeature של הדוגמה. לפעמים ערך הסף קרוב מאוד לערך הfeature של דוגמת המבחן. היינו רוצים להתחשב בערכים "קרובים" לערך הסף בעת סיווג דוגמת מבחן, ולא לחרוץ את גורלה של הדוגמה לתת־עץ אחד בלבד; לצורך כך נציג את האלגוריתם הבא:

יהיו עץ החלטה T, דוגמת מבחן $x \in R^d$, ווקטור $\varepsilon \in R^d$ המקיים $\forall i \in [1,d]: \varepsilon_i > 0$.

כלל אפסילון־החלטה שונה מכלל ההחלטה הרגיל שנלמד בכיתה באופן הבא:

נניח שמגיעים לצומת בעץ המפצל לפי ערכי התכונה i, עם ערך הסף v_i.

אם מתקיים $|x_i - v_i| \leq \varepsilon_i$ אזי ממשיכים **בשני** המסלולים היוצאים מצומת זה, ואחרת ממשיכי לבן המתאים בדומה לכלל ההחלטה הרגיל. לבסוף, מסווגים את הדוגמה x *בהתאם לסיווג הנפוץ ביותר של הדוגמאות הנמצאות בכל העלים אליהם הגענו במהלך הסיור על העץ (במקרה של שוויון – הסיווג ייקבע להיות $True$.)*

יהא T עץ החלטה לא גזום, ויהא T' העץ המתקבל מ־T באמצעות גיזום מאוחר שבו הוסרה הרמה התחתונה של T (כלומר כל הדוגמות השייכות לזוג עלים אחים הועברו לצומת האב שלהם).

הוכיחו\הפריכו: **בהכרח** קיים ווקטור ε כך שהעץ T עם כלל אפסילון־החלטה והעץ T' עם כלל ההחלטה הרגיל יסווגו כל דוגמת מבחן בR^d בצורה זהה.

T' = pruned tree
T = decision tree
Lets think about the bottom split in T:

- The epsilon-decision has to send every example down both branches to simulate pruning.
- But examples can be arbitrarily far from the threshold.
- Therefore, no finite epsilon can cover all examples.

So the statement is false there does not necessarily exist an epsilon that works for all test examples

A counterexample:

1. Take a tiny **tree with 1 split at the bottom**:

Node A (split on x1, threshold v1=0)

/ \

Leaf L1 (label True) Leaf L2 (label False)

- This is the **unpruned tree** T.
- The **pruned tree** T'removes the bottom level:

Node A (leaf, label determined by majority)

- So T'always classifies **all examples as the majority label**.
- Suppose **majority label at Node A** = True

Step 1: Pick a test example

- Let's pick $x_1 = 100$(very far from threshold $v_1 = 0$).

Step 2: Apply epsilon-decision

- To simulate pruning, the **epsilon-decision tree** needs to **follow both branches**, because T'"merges" L1 and L2.
- But for $x_1 = 100$, the distance from threshold is:

$$| x_1 - v_1 | = | 100 - 0 | = 100$$

- To follow both branches, we need $\epsilon_1 \geq 100$.

Step 3: Pick another test example

- Now pick $x_1 = -1000$.
- Then $| x_1 - v_1 | = 1000$.
- To follow both branches for **this example too**, we would need $\epsilon_1 \geq 1000$.

Step 4: Problem

- There is **no finite** $\epsilon_1$that works for **all possible real numbers** $x_1 \in \mathbb{R}$.
- Therefore, you **cannot guarantee** that the epsilon-decision tree will always behave like the pruned tree.

חלק ב' - היכרות עם הקוד

רקע

חלק זה הוא רק עבור היכרות הקוד, עבורו עליו במלואו וודאו כי היכם מבינים את הקוד.
בחלק של הלמידה, נעזר ב-$dataset$, הדאטה חולק עבורכם לשתי קבוצות: קבוצת אימון train.csv וקבוצת מבחן test.csv. בכלל, קבוצת האימון תשמש לבניית המסווגים, וקבוצת המבחן תשמש להערכת ביצועיהם.

בקובץ utils.py תוכלו למצוא את הפונקציות הבאות לשימושכם:
load_data_set, create_train_validation_split, get_dataset_split
אשר טוענות/מחלקת את הדאטה בקבצי ה־csv למערכי np.array (קראו את תיעוד הפונקציות).

הדאטה של 3ID עבור התרגיל מכיל מדדים שנאספו מצילומים שנועדו להבחין בין גידול שפיר לגידול ממאיר.
כל דוגמה מכילה 30 מדדים באלה, ותווית בינארית **diagnosis** הקובעת את סוג הגידול (0=שפיר, 1=ממאיר).
כל התכונות (מדדים) רציפות. העמודה הראשונה מציינת האם האדם חולה (M) או בריא (B). שאר העמודות מציינות תכונות רפואיות שונות של המטופל (התכונות מורכבות ואינכם צריכים להתייחס למשמעותן כלל).

<u>תיקיית $ID3 - dataset$:</u>

- תיקיה זו אלו מכילה את קבצי הנתונים עבור $ID3$.

<u>קובץ $utils.py$:</u>

- קובץ זה מכיל פונקציות עזר שימושיות לאורך התרגיל, כמו טעינה של $dataset$ וחישוב הדיוק.
- בחלק הבא יהיה עליכם לממש את הפונקציות $l2_dist$ ו$accuracy$. קראו את תיעוד הפונקציות ואת ההערות הנמצאות תחת התיאור. **TODO**

<u>קובץ $unit_test.py$:</u>

- קובץ בדיקה בסיסי שיכול לעזור לכם לבדוק את המימוש.

<u>קובץ $DecisionTree.py$:</u>

- קובץ זה מכיל 3 מחלקות שימושית לבניית עץ $ID3$ שלנו.
  - <u>המחלקה $Question$:</u> מחלקה זו מממשת הסתעפות של צומת בעץ. היא שומרת את התכונה ואת הערך שלפיהם מפצלים את הדאטה שלנו.
  - <u>המחלקה $DecisionNode$:</u> מחלקה זו מממשת צומת בעץ ההחלטה.
    הצומת מכיל שאלה $Question$ ואת שני הבנים $true_branch, false_branch$ כאשר $true_branch$ הוא הענף בחלק של הדאטה שעונה $True$ על שאלת הצומת (הפונקציה $match$ של ה $Question$ מחזירה $True$).
    ו־$false_branch$ הוא הענף בחלק של הדאטה שעונה $False$ על שאלת הצומת (הפונקציה $match$ של ה $Question$ מחזירה $False$).
  - <u>המחלקה $Leaf$:</u> מחלקה זו מממשת צומת שהוא עלה בעץ ההחלטה. העלה מכיל לכל אחד מהמחלקות בדאטה את מס' הדוגמאות בעלה עבור כל מחלקה (למשל: $\{'B' : 5, 'M' : 6\}$).

<u>קובץ $ID3.py$:</u>

- קובץ זה מכיל את המחלקה של $ID3$ שתצטרכו לממש חלקים ממנה, עיינו בהערות ותיעוד המתודות.

<u>קובץ $ID3_experiments.py$:</u>

הקובץ מכיל את הניסויים הבאים, שיוסברו בהמשך, ID3 קובץ הרצת הניסויים של:
$cross_validation_experiment, basic_experiment$

חלק ג' – חלק רטוב ID3 (21 נק')

עבור חלק זה מותר לכם להשתמש בספריות הבאות:
All the built in packages in python, sklearn, pandas ,numpy, random, matplotlib, argparse, abc, typing.

<u>אך כמובן שאין להשתמש באלגוריתמי הלמידה, או בכל אלגוריתם או מבנה נתונים אחר המהווה חלק מאלגוריתם למידה אותו תתבקשו לממש.</u>

1. (5 נק') השלימו את הקובץ utils.py ע"י מימוש הפונקציות $l2_dist$ ו $-accuracy$.

   קראו את תיעוד הפונקציות ואת ההערות הנמצאות תחת התיאור. **TODO**

   (הריצו את הטסטים המתאימים בקובץ $unit_test.py$ *לוודא שהמימוש שלכם נכון).*

   שימו לב! בתיעוד ישנן הגבלות על הקוד עצמו, <u>אי־עמידה בהגבלות אלו תגרור הורדת נקודות</u>.

   בנוסף, שנו את ערך הID בתחילת הקובץ מ־123456789 למספר תעודת הזהות של אחד מהמגישים.

2. (16 נק') **אלגוריתם ID3:**
   a. (10 נק') השלימו את הקובץ ID3.py ובכך ממשו את אלגוריתם $ID3$ כפי שנלמד בהרצאה.

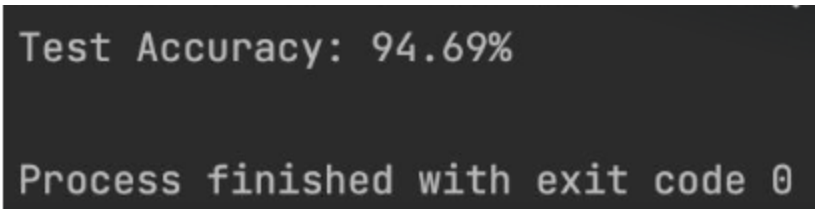
   TODO

   שימו לב:

   - שכל התכונות רציפות.
   - אתם מתבקשים להשתמש בשיטה של חלוקה דינמית המתוארת בהרצאה
   - **כאשר בוחנים ערך סף לפיצול של תכונה רציפה, דוגמאות עם ערך השווה לערך הסף משתייכות לקבוצה עם הערכים הגדולים מערך הסף**.
   - במקרה שיש כמה תכונות אופטימליות בצומת מסוים בחרו את התכונה בעלת <u>האינדקס המקסימלי</u> .
   - כלל המימוש הנ"ל צריך להופיע בקובץ בשם $ID3.py$, באזורים המוקצים לכך.
   - השלימו את הקוד החסר אחרי שעיינתם והפנמתם את הקובץ $DecisionTree.py$ ואת המחלקות שהוא מכיל.

   b. (6 נק') ממשו את $basic_experiment$ שנמצאת ב $ID3_experiments.py$. **TODO**

   והריצו את החלק המתאים ב $main$ ציינו בדו"ח את הדיוק שקיבלתם. ✍



גילוי נאות - שימוש בכלי AI לפתרון תרגיל הבית (5 נק')

כפי שפורט במסמך המדיניות, אנו רואים חשיבות מרכזית בפיתוח **חשיבה ביקורתית** ובשימוש רפלקטיבי בכלי AI. אנו מצפים כי השימוש שתבצעו יהיה לצורך **העצמת הלמידה והבנייה המעמיקה של הידע**, ולא לצורך העתקה עיוורת או הימנעות ממאמץ קוגניטיבי.

בהמשך לכך, בשאלה זו תצטרכו לציין באילו כלי AI השתמשתם, ולפרט **בהרחבה** כיצד השתמשתם בהם, בכל אחד מחלקי התרגיל:

1. **בחלק התיאורטי ("היבש") - (2.5 נק')**
2. **בחלק המעשי ("הרטוב") - (2.5 נק')**

מוזמנים להסתכל במסמך המדיניות על דוגמאות לרמת הפירוט הנדרשת.

בנוסף, בחלק זה, אנו מזמינים אתכם לצרף כנספח את השיחה שלכם עם כלי ה-AI בנוגע לפתרון שיעורי הבית.

שימו לב! סטודנטים שרמת הפירוט שלהם בשאלה זו תהיה "לא משכנעת מספיק", ו/או שלא צירפו את השיחה שלהם עם כלי ה-AI איתו עבדו, יוכלו להיבחר בצורה מדגמית לבחינה קצרה בעל פה על הגיליון והקוד שהגישו.

Dry part:
Question 1 : I gave chat the algorithm of the value and policy iteration with the new reward system and asked him to put the new utility system at the right places in the algorithm.

Question 2 : at the end of the question I got stuck and couldn't understand how to compute the Policy evaluation of π_1, so I asked the help of chat.

Question about the anytime algorithm: I gave chat the idea to calculate until T because then the error would be close to epsilon and when we calculate more T+1 we get even smaller error (anytime) , and I asked him to write the formal algorithm while calculating each episode in a first visit way.

Question on machine learning: I gave chat the counterexample that I want and asked him to write it formally.

can u give me a counter example where T is of depth = 1 and threshold 0 with true and false childen, and T' the prunes tree , and T will use this desicion , and then we will get an x1 that needs epsilon = 100 , and x2 that needs epsilon = 1000 כלל אפסילון־החלטה שונה מכלל ההחלטה הרגיל שנלמד בכיתה באופן הבא:
נניח שמגיעים לצומת בעץ המפצל לפי ערכי התכונה i, עם ערך הסף v_i. אזי ממשיכים בשני המסלולים היוצאים מצומת זה, אם מתקיים |x_i-v_i |≤ε_i ואחרת ממשיכי לבן המתאים בדומה לכלל ההחלטה הרגיל. לבסוף, מסווגים בהתאם לסיווג הנפוץ ביותר של הדוגמאות הנמצאות בכל x את הדוגמה העלים אליהם הגענו במהלך הסיור על העץ (במקרה של שוויון – הסיווג ייקבע להיות

Perfect! Let's construct exactly the **counterexample** you're asking for, step by step.

Step 1: Define the trees

Unpruned tree T (depth = 1):

Wet part:
I did use the Ai mainly for syntax purposes, but since I am retaking the course I didn't need it that much , the algorithms where so close to the last semester so I just modified and improved them but didn't need that much of ai assistant , except for the RL part I needed it to understand the algo and explain it in a simple way so I can program it.

הוראות הגשה

- ✓ הגשת התרגיל תתבצע אלקטרונית בזוגות בלבד.
- ✓ הקוד שלכם ייבדק (גם) באופן אוטומטי ולכן יש להקפיד על הפורמט המבוקש. הגשה שלא עומדת בפורמט לא תיבדק (ציון 0).
- ✓ המצאת נתונים לצורך בניית הגרפים אסורה ומהווה עבירת משמעת.
- ✓ הקפידו על קוד קריא ומתועד. התשובות בדוח צריכות להופיע לפי הסדר.
- ✓ יש להגיש קובץ zip יחיד בשם `AI3_<id1>_<id2>.zip` (ללא סוגריים משולשים) שמכיל:

- קובץ בשם `AI_HW3.PDF` המכיל את תשובותיכם לשאלות היבשות.
- קבצי הקוד שנדרשתם לממש בתרגיל **<u>ואף קובץ אחר:</u>**
  - קובץ – utils.py
  - בחלק של עצי החלטה – ID3.py, ID3_experiments.py
  - בחלק של mdp ו־RL – mdp_rl_implementation.py